

IPv6

Note: Like the other flagged topics, IPv6 isn't covered in your source notes — drafted fresh from general SDE-interview networking knowledge.

1. Why IPv6 Exists

IPv4's 32-bit address space caps out at ~4.3 billion addresses — a number that seemed enormous in the 1980s and is now inadequate for a world of billions of phones, IoT devices, and servers, especially once NAT/CIDR's mitigations (covered in earlier topics) started running into their own limits. **IPv6 is the actual long-term fix**, not a patch: a **128-bit** address space.

IPv4: $2^{32} \approx 4.3 \times 10^9$ addresses
IPv6: $2^{128} \approx 3.4 \times 10^{38}$ addresses (enough for roughly 340 undecillion addresses)

Interview soundbite: "NAT and CIDR are both explicitly *short-term* mitigations for IPv4 exhaustion — they reduce waste and extend the runway, but the address space itself is still fundamentally capped at 2^{32} . IPv6 doesn't optimize around that limit; it removes it, by using an address space so large that exhaustion isn't a realistic concern on any practical timescale."

2. Address Format

- **128 bits**, written as **8 groups of 4 hexadecimal digits**, separated by colons:

```
2001:0db8:85a3:0000:0000:8a2e:0370:7334
```

- **Leading zeros in each group can be omitted:**

```
2001:db8:85a3:0:0:8a2e:370:7334
```

- **One (and only one) run of consecutive all-zero groups can be compressed to `::`:**

```
2001:db8:85a3::8a2e:370:7334
```

Why only once per address? If `::` were allowed to appear twice, the address would become ambiguous — there'd be no way to tell how many zero-groups each `::` was supposed to represent. Restricting it to a single occurrence keeps the notation unambiguous and fully reversible back to the expanded form.

Loopback: `::1` (equivalent to IPv4's `127.0.0.1`).

Unspecified address: `::` (all zeros — used, e.g., as a source address before a device has configured one).

3. Address Types — no more broadcast

IPv6 has **unicast**, **multicast**, and **anycast** — but **no broadcast** at all (a deliberate removal, not an oversight).

Type	Meaning
Unicast	One sender, one specific receiver (same as IPv4)
Multicast	One sender, a <i>group</i> of interested receivers (IPv4 has this too)
Anycast	One address shared by <i>multiple</i> devices; a packet sent to it is delivered to whichever one is topologically nearest to the sender
~~Broadcast~~	Removed entirely — replaced by more targeted multicast

Why remove broadcast? Broadcast traffic is inherently wasteful — every device on the local segment must process it, whether interested or not, and broadcast storms (recall the Spanning Tree topic) are a real operational hazard. IPv6 replaces every use case broadcast used to serve (e.g., "find all routers on this link") with specific, well-defined **multicast groups** — e.g., the "all routers" multicast address — so only devices that actually care need to process the packet, while still achieving the same practical goal.

Anycast is a genuinely new capability IPv4 doesn't cleanly offer — used heavily in practice for things like DNS root server addresses and CDN edge nodes, where you want a request routed to the *nearest* available instance of a service, without the client needing to know which specific instance that is.

4. Header Simplification

IPv6 deliberately **simplifies and fixes the size** of the base header, moving optional complexity out into separate, chainable **extension headers**.

IPv6 base header (fixed, 40 bytes)

Next Header field → points to first extension header (if any)

[Extension Header 1] → Next Header → [Extension Header 2] → ... → [Actual payload/data]

What was removed from the IPv4 header (and why)

- **Header checksum — removed entirely.** Reasoning: both the Data Link layer (frame-level CRC) and the Transport layer (TCP/UDP checksums) already check for corruption; requiring every router to recompute *another* checksum at every hop (as IPv4 does, because of the TTL field changing) was judged to be redundant overhead for little actual benefit, given those other layers already do this job.
- **Fragmentation fields (moved) — routers no longer fragment packets at all.** In IPv6, **only the original source host** is permitted to fragment a packet (via an extension header, if truly needed) — an intermediate router that receives a too-large packet must simply **drop it** and send back an ICMPv6 "Packet Too Big" message, rather than fragmenting it itself. This pushes the entire practical burden toward **Path MTU Discovery** (the same idea introduced in the IPv4 Fragmentation topic) being the *expected*, primary mechanism, not a rarely-used optimization.
- **Header Length field — removed**, since the base header is now a fixed 40 bytes; there's nothing variable-length left in it to measure.

Interview soundbite: "IPv4's header tries to be one-size-fits-all, with optional fields and variable length baked directly into the base header that every router has to parse. IPv6 flips this: the base header is small and fixed, and every non-essential piece of complexity — fragmentation info, routing options, security — becomes an optional extension header that a router only needs to look at if it's actually relevant, and can otherwise skip right past."

5. Address Autoconfiguration — SLAAC

IPv6 devices can configure their own address **without a DHCP server at all**, using **SLAAC (StateLess Address AutoConfiguration)**:

1. A device generates a **link-local address** for itself, typically derived from its own MAC address (or a randomized interface identifier, for privacy).
 2. It listens for (or solicits) a **Router Advertisement** from a local router, which announces the network's prefix.
 3. The device combines the announced prefix with its own interface identifier to form a complete, globally-routable address — no central server needed to hand out addresses one at a time.
- DHCPv6 (a IPv6-specific evolution of DHCP) still exists for cases needing more centralized control (specific address assignment, additional configuration like DNS servers) — SLAAC and DHCPv6 can coexist, addressing different needs.
-

6. NDP replaces ARP

IPv6 has **no ARP at all** — ARP's job (map an IP address to a MAC address on the local link) is folded into **NDP (Neighbor Discovery Protocol)**, which is built on top of ICMPv6 messages rather than being its own separate Data-Link-adjacent protocol:

ARP concept (IPv4)	NDP equivalent (IPv6)
ARP Request/Reply	Neighbor Solicitation / Neighbor Advertisement
(no direct equivalent — router discovery was manual/DHCP-driven)	Router Solicitation / Router Advertisement (also powers SLAAC)
Gratuitous ARP (conflict detection)	Duplicate Address Detection (DAD)

Why fold this into ICMPv6 rather than keeping a separate ARP-like protocol?

Consolidating these control-plane functions into one consistent, extensible messaging framework (ICMPv6) rather than maintaining several separate, differently-structured protocols (as IPv4 does with ARP, ICMP, and IGMP as distinct entities) simplifies the overall design and makes it easier to extend consistently.

7. Coexisting with IPv4 — Transition Mechanisms

Since the entire Internet couldn't switch to IPv6 overnight, several practical coexistence strategies exist:

- **Dual stack** — a device/router runs **both** IPv4 and IPv6 simultaneously, using whichever the destination supports. The most common, straightforward approach during the (very long) transition period.
- **Tunneling** — IPv6 packets are encapsulated inside IPv4 packets to cross IPv4-only network segments (and vice versa), letting isolated "islands" of one protocol communicate across infrastructure that only understands the other.
- **NAT64 / Translation** — actively translates between IPv6 and IPv4 packets at a gateway, allowing an IPv6-only client to reach an IPv4-only server (or vice versa) without either end needing to run both stacks itself.

Interview soundbite: "There was never going to be a single flag-day cutover for something as large as the entire Internet — dual stack, tunneling, and translation are three different pragmatic answers to the same underlying question: how do two networks speaking different IP versions still talk to each other during a transition that's taken (and will keep taking) decades?"

Summary: IPv4 vs IPv6

Aspect	IPv4	IPv6
Address length	32 bits	128 bits
Address space	~4.3 billion	$\sim 3.4 \times 10^{38}$
Notation	Dotted decimal	Colon-separated hex, with <code>::</code> compression
Header	Variable length (20–60 bytes), includes checksum	Fixed 40 bytes, no checksum, extension headers for extras
Fragmentation	Performed by any router along the path	Only by the source host; routers drop and signal "too big" instead
Address assignment	Manual, or DHCP	SLAAC (stateless) or DHCPv6 (stateful)
Broadcast	Yes	Removed — replaced by multicast
Address resolution	ARP (separate protocol)	NDP (built on ICMPv6)

Interview Questions With Answers

Q1. Why is IPv6's 128-bit address space considered a genuine long-term fix, while NAT and CIDR are described as short-term mitigations?

Answer: NAT and CIDR both work *within* IPv4's fixed 32-bit address ceiling — NAT lets many private hosts share few public addresses, and CIDR allocates that fixed space more efficiently, but neither changes the fact that there are still only ~4.3 billion possible addresses total, which is already insufficient for the number of connected devices in the world. IPv6's 128-bit space is roughly 3.4×10^{38} addresses — large enough that address exhaustion isn't a realistic concern on any practical planning horizon, so it removes the underlying constraint rather than optimizing around it.

Q2. Why is the `::` compression notation only allowed to appear once in an IPv6 address?

Answer: `::` represents "however many all-zero groups are needed to pad the address out to its full 8 groups" — if it were allowed twice, there would be no way to determine how many zero-groups each occurrence represents individually, making the address ambiguous and impossible to reliably expand back to its full 128-bit form. Restricting it to a single occurrence per address guarantees the compressed notation always has exactly one unambiguous full expansion.

Q3. Why did IPv6 remove broadcast addressing entirely?

Answer: Broadcast traffic requires every device on a local network segment to receive and process it regardless of whether they actually care about the message, which is inherently wasteful and creates operational risks (broadcast storms). IPv6 replaces every practical use case broadcast used to serve with specific, purpose-defined multicast groups (e.g., an "all routers" multicast address) — achieving the same functional goals (like discovering routers, or address resolution via NDP) while only devices that are actually members of the relevant multicast group need to process the traffic.

Q4. Why was the header checksum removed from IPv6, given that IPv4 keeps it?

Answer: The header checksum in IPv4 exists to protect the header from corruption, but this protection is largely redundant with checks already performed elsewhere — the Data Link layer typically includes its own frame-level error detection (like CRC), and the Transport layer (TCP/UDP) has its own checksum covering its own header and data. Since IPv4 needs to recompute this checksum at every single router hop anyway (because the TTL field changes at every hop), it was judged to be meaningful per-hop overhead for benefit that's largely duplicated by layers above and below it — IPv6 removes it and relies on those other layers' error detection instead.

Q5. Why can only the source host fragment an IPv6 packet, rather than any router along the path (as in IPv4)?

Answer: This is a deliberate design trade-off to reduce the processing burden on routers — fragmentation is comparatively expensive work (splitting a packet, computing new headers/offsets for each piece), and requiring every router to be capable of doing this on demand adds complexity and per-hop cost to core network infrastructure. By restricting fragmentation to only the source host (which can use Path MTU Discovery to learn the right size *before* sending), routers are simplified: if a packet is too large for a given link, the router simply drops it and reports back with an ICMPv6 "Packet Too Big" message, pushing the responsibility for correctly sizing packets to the edge, where it's cheaper to handle repeatedly.

Q6. What is SLAAC, and what problem does it solve compared to needing a DHCP server?

Answer: SLAAC (StateLess Address AutoConfiguration) lets an IPv6 device configure its own globally-routable address without any centralized server handing out addresses — the device generates its own interface identifier, listens for a Router Advertisement announcing the local network's prefix, and combines the two to form a complete address itself. This removes the dependency on a DHCP server being present, reachable, and correctly configured just for basic address assignment — useful in networks where deploying and maintaining a DHCP server is unnecessary overhead, though DHCPv6 remains available for cases still needing centralized control (like handing out specific addresses or additional configuration details).

Q7. How does NDP replace both ARP and part of what DHCP/router-discovery does in IPv4?

Answer: NDP consolidates several related "who/what is on my local link" functions into one ICMPv6-based framework: Neighbor Solicitation/Advertisement messages replace ARP's Request/Reply for mapping IP addresses to MAC addresses, Duplicate Address Detection replaces gratuitous ARP's conflict-checking role, and Router Solicitation/Advertisement messages (which also power SLAAC) provide a standardized way to discover routers and network prefixes — something IPv4 handled more ad hoc via a mix of ARP, DHCP, and manual configuration, rather than one unified protocol.

Q8. Why are dual stack, tunneling, and NAT64 all necessary, rather than just picking one universal transition strategy?

Answer: Different points in the network face different constraints during the IPv4-to-IPv6 transition. Dual stack works well when a device/network can afford to run both stacks simultaneously and simply prefer whichever the destination supports. Tunneling is needed when an IPv6-capable "island" needs to communicate across infrastructure that genuinely only understands IPv4 (or vice versa), without every intermediate hop needing to be upgraded. NAT64/translation is needed specifically when one endpoint is IPv6-only and the other is IPv4-only, with no shared protocol between them at all, requiring active translation rather than just encapsulation. Since real-world networks encounter all three of these situations simultaneously across a transition spanning decades, no single mechanism covers every case.

Q9. Scenario: A company runs a mix of legacy IPv4-only internal servers and new IPv6-only client devices, and a client needs to reach one of the legacy servers.

Which transition mechanism is most directly applicable, and why not dual stack?

Answer: NAT64 (translation) is the most directly applicable mechanism here — the IPv6-only client and the IPv4-only server share no common protocol at all, so something must actively translate packets between IPv6 and IPv4 at a gateway sitting between them, typically paired with DNS64 to synthesize an IPv6-reachable address representing the IPv4-only server. Dual stack isn't the fix here specifically because the *client* is described as IPv6-only (not running both stacks) — dual stack would require upgrading the client itself to also support IPv4, which may not be feasible or desired; NAT64 solves the problem without requiring any change to either the already-IPv6-only client or the legacy IPv4-only server.